Autoencoders (AEs) and Variational Autoencoders (VAEs)

Unsupervised Representation Learning Models

Week 10 Contents

Unsupervised Learning:

- Autoencoders
  - Architecture of Autoencoders
  - Reconstruction Loss Function (MSE in detail, Binary Cross-Entropy: Discussion)
  - Types of Autoencoders (Simple, Deep, CNN-based)
  - Limitations of Standard Autoencoders
- Variational Autoencoders
  - VAE Architecture and Reparameterization Trick
  - Loss Function
- Hands-on Implementation of Autoencoders

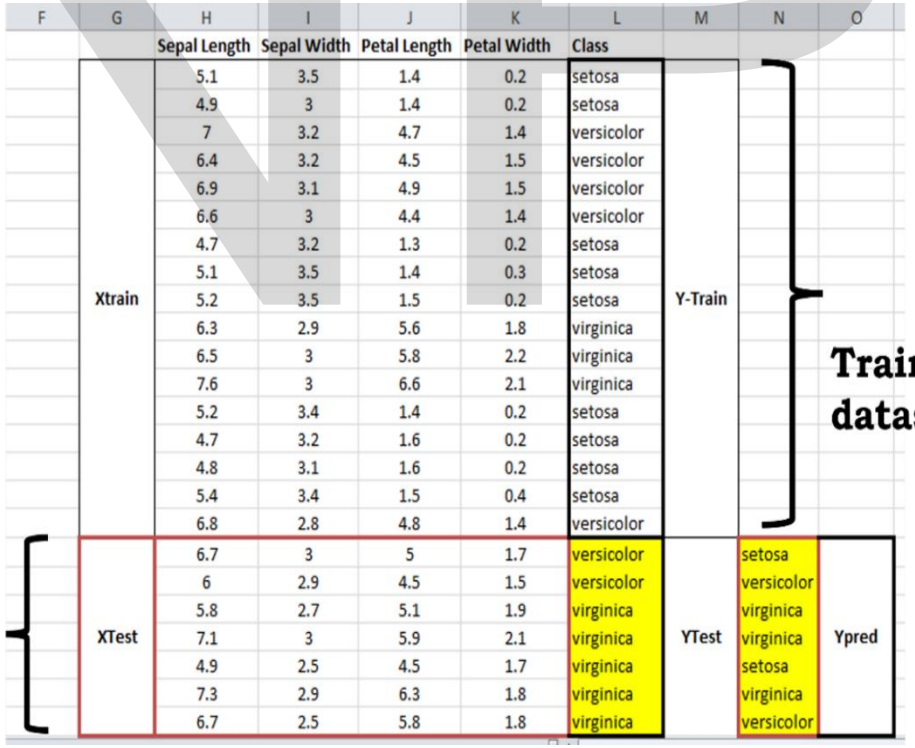
Today's Session Overview

- Autoencoders
  - Introduction
  - Architecture of Autoencoders
  - Reconstruction Loss Function (MSE, Binary Cross-Entropy)

Introduction

Most traditional neural network architectures such as ANNs, CNNs, and RNNs are primarily used in supervised learning, where:

- Each **input sample** is associated with an **explicit target label**
- The network learns a mapping from **input → output**
- **Training minimizes the error** between predicted output and true label



	Sepal Length	Sepal Width	Petal Length	Petal Width	Class			
	5.1	3.5	1.4	0.2	setosa			
	4.9	3	1.4	0.2	setosa			
	7	3.2	4.7	1.4	versicolor			
	6.4	3.2	4.5	1.5	versicolor			
	6.9	3.1	4.9	1.5	versicolor			
	6.6	3	4.4	1.4	versicolor			
	4.7	3.2	1.3	0.2	setosa			
	5.1	3.5	1.4	0.3	setosa			
Xtrain	5.2	3.5	1.5	0.2	setosa	Y-Train		
	6.3	2.9	5.6	1.8	virginica			
	6.5	3	5.8	2.2	virginica			
	7.6	3	6.6	2.1	virginica			
	5.2	3.4	1.4	0.2	setosa			
	4.7	3.2	1.6	0.2	setosa			
	4.8	3.1	1.6	0.2	setosa			
	5.4	3.4	1.5	0.4	setosa			
	6.8	2.8	4.8	1.4	versicolor			
	6.7	3	5	1.7	versicolor		setosa	
	6	2.9	4.5	1.5	versicolor		versicolor	
	5.8	2.7	5.1	1.9	virginica		virginica	
XTest	7.1	3	5.9	2.1	virginica	YTest	virginica	Ypred
	4.9	2.5	4.5	1.7	virginica		setosa	
	7.3	2.9	6.3	1.8	virginica		virginica	
	6.7	2.5	5.8	1.8	virginica		versicolor	

Training dataset

Testing dataset

But what if labels are not available?

Age (years)	Height (cm)	Weight (kg)
6	115	20
8	125	25
10	135	30
12	145	36
14	155	45
16	165	55
18	170	62
20	172	66
22	174	69
25	175	72
30	176	???

This leads to **unsupervised learning: the model is trained only on data without explicit labels.**

How does the network learn?

In unsupervised learning:

- The model learns by discovering patterns, structures, and correlations within the data.
- The objective is not classification or prediction, but **structure discovery.**

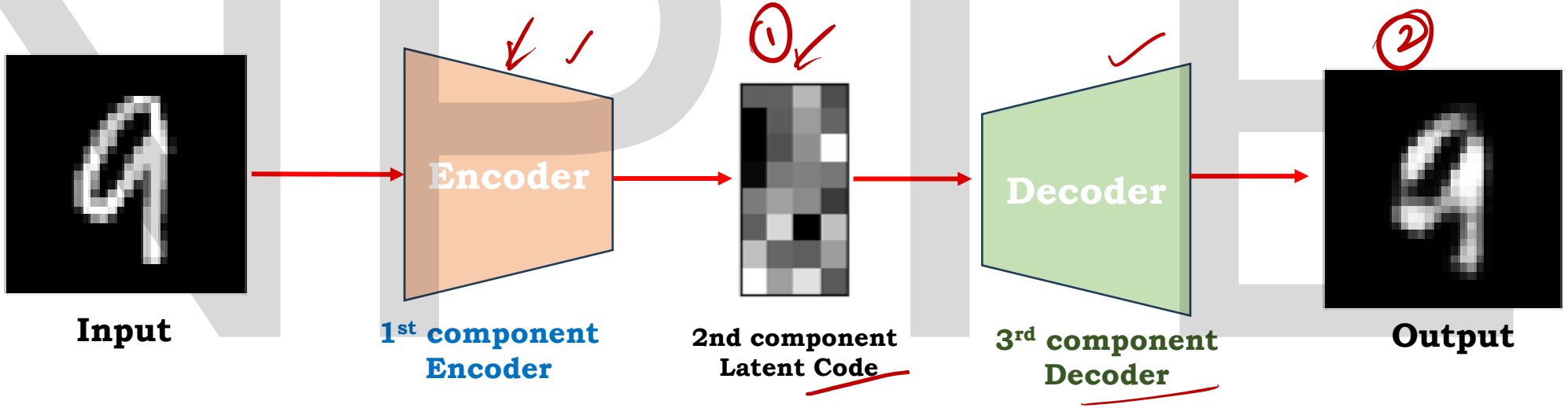
While unsupervised learning broadly focuses on discovering patterns and structure in unlabelled data, a special class of deep learning models aims to ***learn meaningful representations* of the data itself.**

Autoencoders belong to this class, where the objective is not classification or prediction, but representation learning through reconstruction.

Architecture of AutoEncoder

Autoencoders have two objectives:

1. The primary objective is to learn a compact latent representation of the input through the encoder, enabling unsupervised feature learning.
2. The second objective is to reconstruct the input from this latent representation using the decoder.

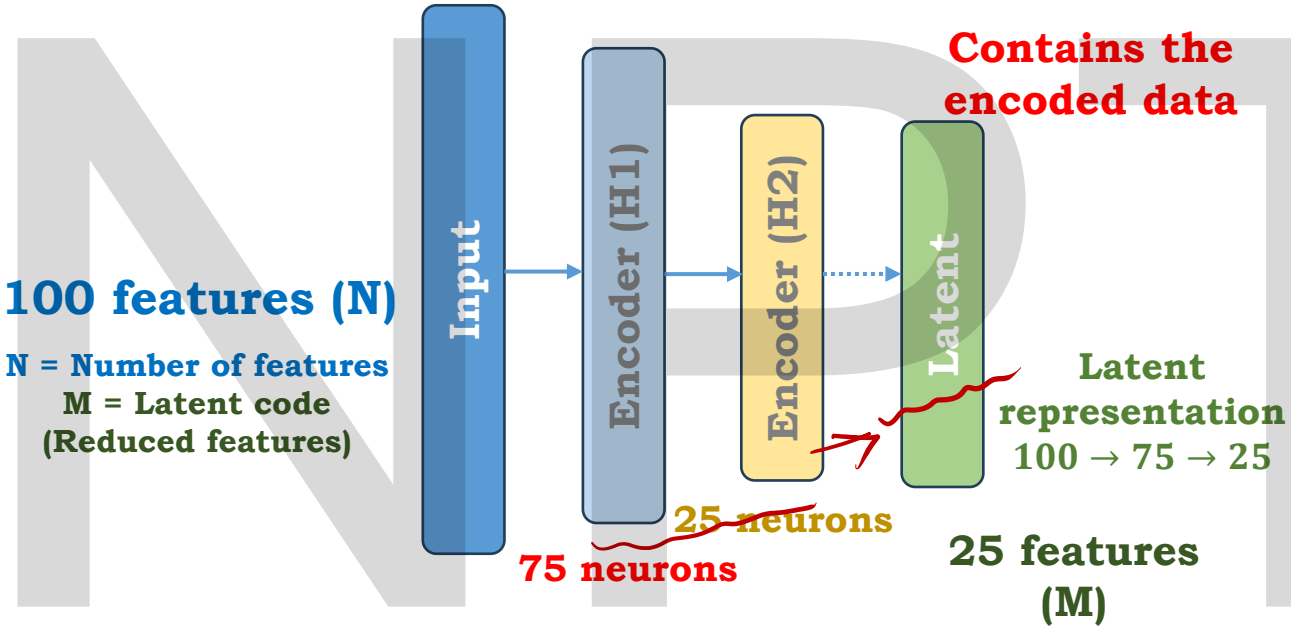


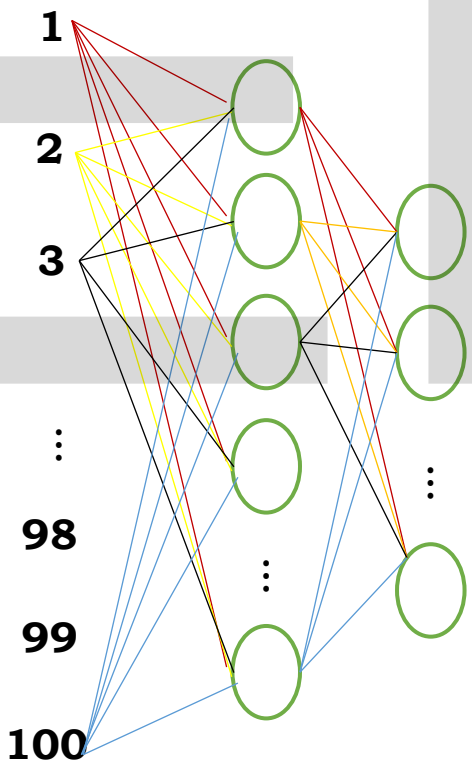
Latent code is a low-dimensional representations learned by the encoder model that captures the essential structure of the input data, enabling efficient reconstruction and generalization.

All 75 values go to all 25 neurons.
Each neuron produces **one output value**.
So, this layer outputs **25 values**.
These **25 values form the latent representation** — a **compact representation** of the original **100 input features**.

Encoder maps the input data → lower-dimensional latent representation by progressively reducing the number of hidden units.

This dimensionality reduction forces the encoder network to learn compact, informative representations of the input data by preserving the most significant structure.
Thus the 1st objective is achieved.

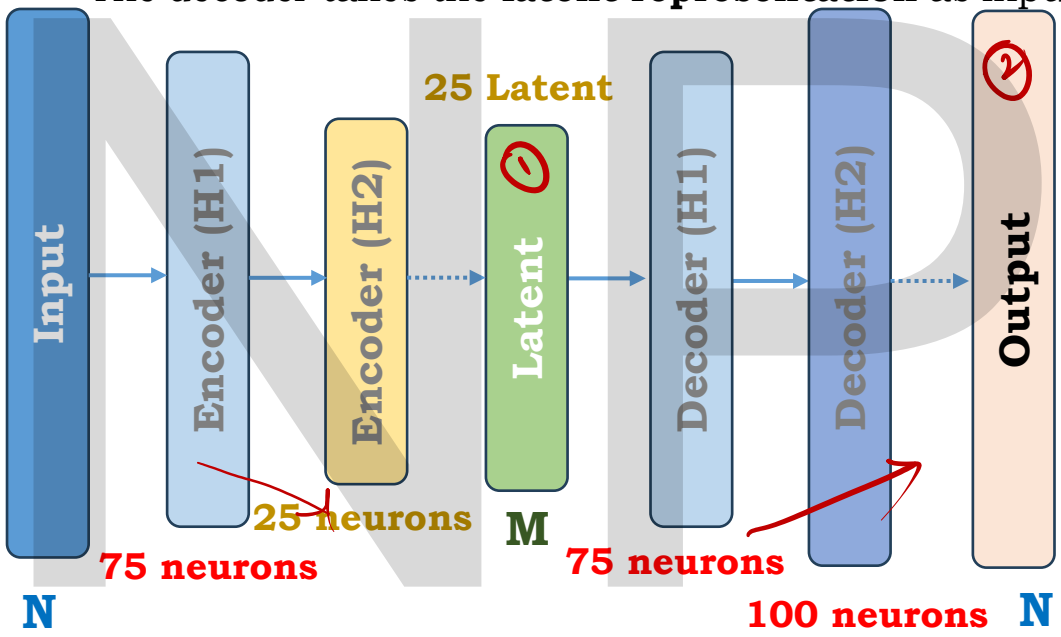




Input: 100 features
Hidden layer: 75 neurons
Each neuron combines all 100 input features and produces **one output value**.
Therefore, the layer outputs **75 values**.
These 75 values are **newly learned representations** from the original 100 features.
Hence, the representation changes from **100 → 75** by **compressing information**, not by deleting features.

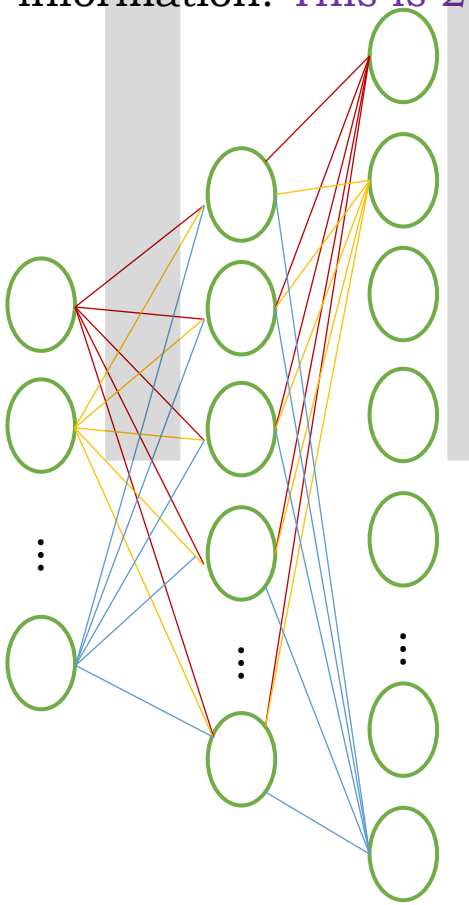
All 75 values go to all 100 output neurons.
Each neuron produces **one output value**.
So, the decoder outputs **100 values**.
These **100 values form the reconstructed output**, matching the dimensionality of the original input.
The reconstruction is an **approximation**, because the input was **compressed** in the encoder.

The decoder takes the **latent representation** as input



All 25 latent values go to all 75 neurons.
Each neuron produces **one output value**.
So, this layer outputs **75 values**.
These 75 values are learned representations that expand and reorganize the information present in the 25 latent variables.
Hence, the decoder performs an **expansion from 25 → 75**.

Decoder maps lower-dimensional latent representation → back to the original input space.
by progressively increasing the number of hidden units.
This dimensionality expansion enables the network to reconstructs the input using only the information preserved by the encoder. Decoder does not recover the original features explicitly.
It reconstructs an approximation of the input based on the information preserved. Therefore, decoder does not create new information. This is 2nd objective.



If $M < N \Rightarrow$ *Undercomplete*
The training process will be completed in the finite time.

If $M > N \Rightarrow$ *Overcomplete*
Training never stops here.
This is used very less in the real world.

Linear Autoencoder: Encoder and Decoder process

$\boldsymbol{H = X \times W_e}$

Where,

H is the encoded data

X is the input

W_e is Encoder Matrix

$H = X \times W_e$

1000 × 3 | ???? → 6 × 3

1000 × 6

W_e is transforming the features in R6 to R3, this is performing the encoding process

$\boldsymbol{\hat{X} = f(H \times W_d)}$

Where,

H is the encoded data

$\hat{X}$ is the reconstructed features

W_d is Decoder Matrix

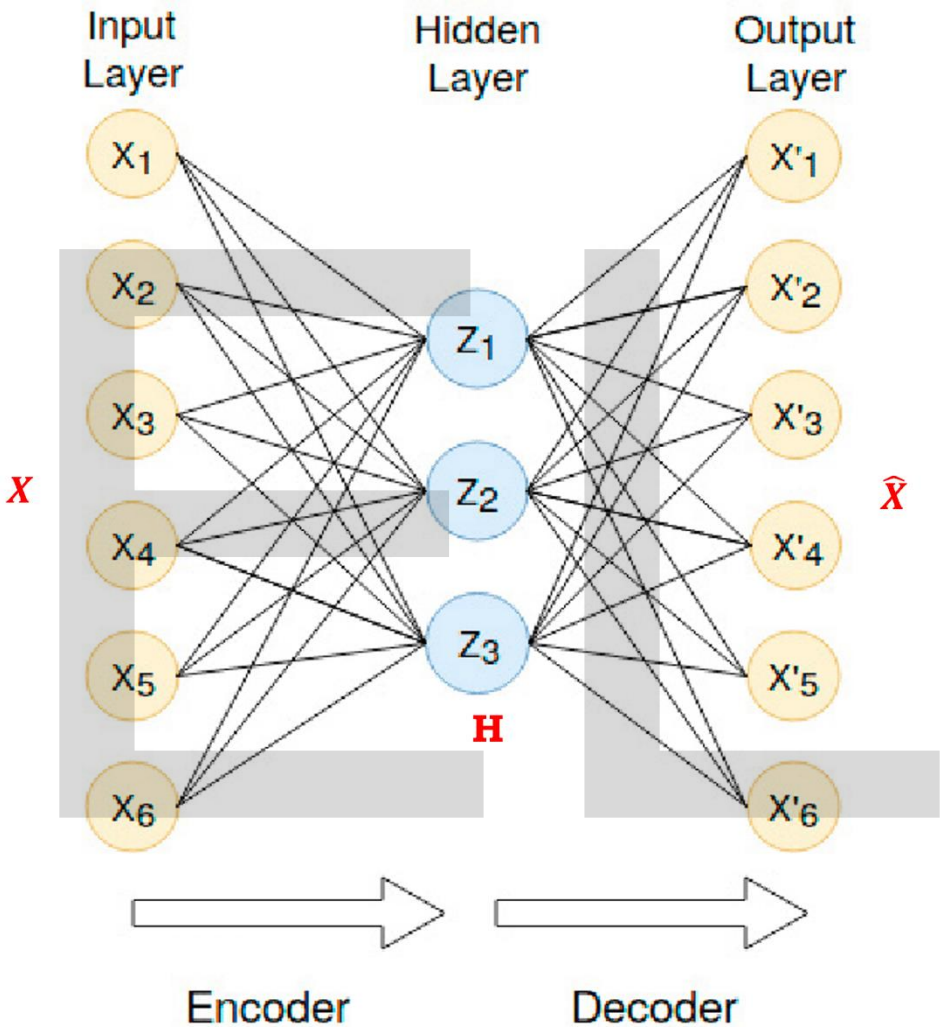
$f(\cdot)$ is output activation

$\hat{X} = H \times W_d$

1000 × 6 | ???? → 3 × 6

1000 × 3

W_d is transforming the features in R3 to R6, this is performing the decoding process.



W_e = 6 × 3

W_d = 3 × 6

Shape is Reversed.

Values can be anything.

To achieve the encoding and decoding process, both W_e and W_d are **learned during training**, which increases the number of parameters and training time.

This is called the **Generic Setting**.

Only W_e to learn and the other is the Transpose of W_e

This is a commonly used and computationally efficient approach. **This is tied-weights assumption ($w_d = w_e^T$).**

Reconstruction Loss Function in AE (MSE)

The main idea is to reconstruct $\hat{X}$ at the end.
Ideal case is $\hat{X} = \mathrm{X}$, **But in the practical case:** $\hat{X} \approx \mathrm{X}$

The basic loss is to minimize the difference between Input X and the reconstructed $\hat{X}$

The loss is typically averaged over all samples:

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n} \| X_i - \hat{X}_i \|^2$$

$l_i = \| X_i - \hat{X}_i \|^2$ Per-sample loss

$\mathrm{f(w)} = min_W \left\| X - \hat{X} \right\|^2$ Get W which minimizes $\left\| X - \hat{X} \right\|^2$

$W_e = \mathrm{W}$
$W_d = W^T$,
where W is the learnable quantity

$f(w) = min_W \|X - HW_d\|^2 \quad \hat{\boldsymbol{X}} = \boldsymbol{HW_d}$

$\mathrm{f(w)} = min_W \|X - HW^T\|^2 \quad W_d = W^T$

$\mathrm{f}(w) = min_W \|X - XW_eW^T\|^2 \quad \boldsymbol{H = XW_e}$

$\mathrm{f(w)} = min_W \|X - XWW^T\|^2 \quad W_e = \mathrm{W}$

$\mathbf{f(W)} = min_W \left\| X - XWW^T \right\|^2$ **Reconstructed error**

$\mathbf{f(W)} = min_W \left\| X - XWW^T \right\|^2 + \lambda \|W\|^2$

Regularization term

The value in the W matrix can be small or large. But always ensure W takes a small value. Therefore, Regularization term.

Once the training process is completed, we obtain the optimized weight matrix W^* ,which minimizes the reconstruction loss.

Using the optimized weights, at encoder

$$XW^* = H^*$$

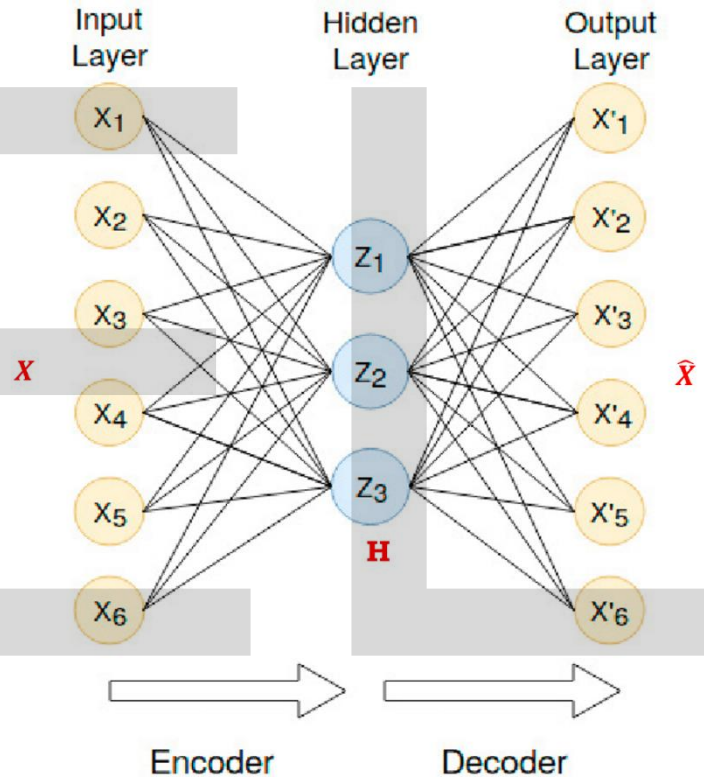
Thus, the latent representation H^*captures the **most rich informative structure of the data required for reconstruction**, not an exact copy of the input.

The reconstructed input is then obtained as:

$$H^* \cdot W^{*T} = \hat{X}$$

$$\| X - \hat{X} \|^2$$

the reconstructed output $\hat{X}$ is **a close approximation of the original input** X.



Reconstruction Loss Function in AE (BCE)

$$l_i = -[x_i \log(\hat{x}_i) + (1-x_i)\log(1-\hat{x}_i)]$$

x_i: 2^p ; $\hat{x}_i$: reconst

$$L_{BCE} = \frac{-1}{n}[\quad]$$

Encode: $H = X \cdot W_e$ $\quad W_e = W$

Dec: $\hat{X} = \sigma(H \cdot W_d)$ $\quad W_d = W^T$

$$f(W) = \min_W \left[L_{BCE}(X, \hat{X}) \right] + \lambda \|W\|^2$$

Next Session

Types of Autoencoders: A Practical Approach

- Shallow Autoencoder
- Deep Autoencoder
- CNN Autoencoder
- Denoising Autoencoder

Limitations of Autoencoders

Thank you